checking for JavaScript libraries?"
- "Explain the difference between module and global declaration files, and describe scenarios where each approach is most appropriate."
- "What are the practical steps involved in manually creating a declaration file for a third-party JavaScript library that lacks builtin TypeScript support?"
- "What are the benefits and trade-offs of shipping declaration files with a library versus including TypeScript source code directly in the package?"
- "In what ways do declaration files encourage best practices and improve code consistency across development teams?"

3.10 Jane's TypeScript interview skill assessment

TypeScript interview challenges often extend beyond syntax, emphasizing a candidate's ability to solve practical problems while ensuring type safety and utilizing advanced typing techniques. These exercises assess how effectively developers can integrate TypeScript in dynamic environments, particularly when working with third-party JavaScript APIs lacking built-in type definitions.

Scenario: Jane was presented with a React code snippet using the YouTube API, which, while functionally correct at runtime, triggered several TypeScript errors related to missing properties on the global window object and implicitly typed parameters. Listing 3.20 lists the code snippet.

Listing 3.20 YouTube API Demo with Type Issues

```
import { useEffect, useRef } from 'react';
const YouTubeApiExample: React.FC = () => {
  const playerRef = useRef<HTMLDivElement>(null); #A
  useEffect(() => {
    const tag = document.createElement('script'); #B
    tag.src = 'https://www.youtube.com/iframe_api';
    const firstScriptTag = document.getElementsByTagName('script'
    firstScriptTag.parentNode!.insertBefore(tag, firstScriptTag);
    window.onYouTubeIframeAPIReady = () => { #D
      if (playerRef.current) { #E
        new window.YT.Player(playerRef.current, { #F
```